

Practical 6

AIM: Implementation of chain matrix multiplication using dynamic programming.

1 ALGORITHM:

Algorithm:-
Input: number of matrices n .
 dimensions of each matrix in the form of an array $P[]$.
 If there are n matrices; $P[]$ contains $n+1$ dimensions.
Output: Display the minimum no. of scalar multiplications required.
Step 1:- Start
Step 2:- Read the number of matrices n .
Step 3:- Store dimensions of matrices in array $P[]$.
Step 4:- Create a table $m[i][j]$ where:
 $m[i][j]$ = minimum cost of multiplying A_i through A_j .
Step 5:- Consider chains of length 2, 3, ..., n .
Step 6:- For every chain $A_i \dots A_j$ try every possible split k .
 $(A_i \dots A_k) \times (A_{k+1} \dots A_j)$
 Calculate $\text{cost} = m[i][k] + m[k+1][j] + P[i-1] \times P[k] \times P[j]$.
Step 7:- Store minimum cost in $m[i][j]$.
Step 8:- Finally $m[1][n]$ gives the minimum no. of scalar multiplications.

2 PROGRAM:

```
#include <iostream>
#include <climits>
using namespace std;

void printTable(int m[][100], int n) {
    cout << "\nDP Table (Minimum Costs):\n";
    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            if (i > j) cout << "   -   ";
            else cout << m[i][j] << "\t";
        }
        cout << endl;
    }
}

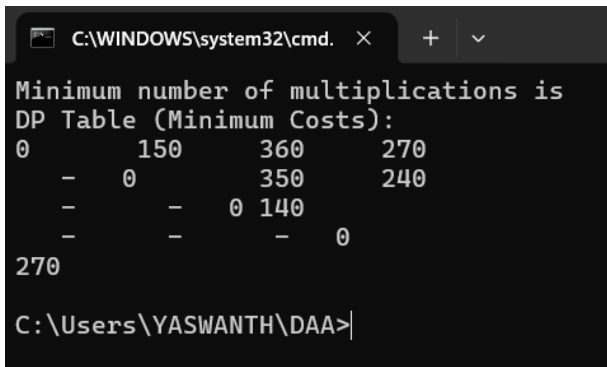
int matrixChainOrder(int p[], int n) {
    int m[100][100];
    for (int i = 1; i < n; i++)
        m[i][i] = 0;
    for (int L = 2; L < n; L++) {
        for (int i = 1; i < n - L + 1; i++) {
            int j = i + L - 1;
            m[i][j] = INT_MAX;
            for (int k = i; k < j; k++) {
                int q = m[i][k] + m[k+1][j] + p[i-1]*p[k]*p[j];
                if (q < m[i][j])
```

```

        m[i][j] = q;
    }
}
printTable(m, n);
return m[1][n-1];}
int main() {
    int arr[] = {3, 5, 10, 7, 2};
    int n = sizeof(arr)/sizeof(arr[0]);
    cout << "Minimum number of multiplications is "
        << matrixChainOrder(arr, n) << endl;
    return 0;}

```

3 OUTPUT:



```

C:\WINDOWS\system32\cmd. X + v
Minimum number of multiplications is
DP Table (Minimum Costs):
0      150      360      270      270
-      0        350      240      270
-      -        0      140      270
-      -        -       0      270
270
C:\Users\YASWANTH\DAA>

```

4 TIME COMPLEXITY:

time complexity

for len = 2
 for i = 1 to n-length-1 → n times
 for j = i to n-length-1 → n times
 for k = i to j-1 → n times
 so approximately
 $n \times n \times n = n^3$
 Therefore Time complexity $O(n^3)$ → best case.

Loop 1 → length = 2 to n → n
 Loop 2 → i = 1 to n-length-1 → n
 Loop 3 → k = i to j-1 → n
 $n \times n \times n = n^3$
 no. of operations = $n \times n \times n = n^3$
 So, time complexity is same for all cases.
 Best case = worst case = average case = n^3 .

5 Conclusion:

Matrix Chain Multiplication using Dynamic Programming efficiently finds the minimum cost of multiplying a chain of matrices by solving subproblems optimally.